NEXUS

INFRASTRUCTURE ENGINEERING
DOCUMENT NX-CH-001
CLASSIFICATION · INTERNAL

DEVELOPER OPERATIONS PLATFORM

Project NEXUS
Engineering Challenge

Engineering the platform behind every modern application.

Engineering Challenge Handbook

ISSUED BY THE PLATFORM RELIABILITY GROUP

Version 1.0
Revision 2026.08
Status Released

PART ONE · THE PROBLEM
SECTION 1.1

The system and the platform

This document explains one problem and describes one system. Everything in it is either background you need, or behaviour the thing you build must have.

The company runs more than **40 services**. Each one is released on its own schedule. Together they handle about **9 million pieces of background work a day** and about **180 releases a week**. **One engineer is on call** at any given moment, and it is usually not the person who wrote the part that is failing.

One customer action, such as placing an order, passes through many of these services. Each has its own database, its own ways of failing, and its own clock. Most of the time this works fine. But operators do not live in “most of the time”. They live in the afternoon when a worker quietly stops picking up work, nobody notices for ninety minutes, and clearing the backlog afterwards breaks something else.

NEXUS is the internal platform that sits underneath all of these services. It moves work between them, carries new versions into them, watches what happens next, restarts things that die, and records enough that a human can piece an incident together afterwards. It makes five promises to the engineers who rely on it. If it breaks one of them, that is an incident, even if no customer noticed.

THE PROMISE	WHAT IT MEANS
01 Work is never lost	If a service gives NEXUS a piece of work and NEXUS accepts it, that work is either finished, or put somewhere visible where an operator can find it. It is never thrown away quietly, and never stuck where nobody checks.
02 Releases can be undone	Any release NEXUS pushes out can be taken back without a human working out by hand what the previous version was. Undoing is a normal action, not an emergency.
03 Disagreements are noticed	When two parts of the system hold different values for the same thing, the platform notices and says so, instead of letting one side quietly win.
04 Failure has a floor	Something that dies gets restarted. Something that cannot come back is taken out of service and reported loudly, instead of being restarted forever.
05 The past can be reconstructed	For any recent moment, an operator can find out what the platform was doing, what it had been asked to do, and what it decided.

NEXUS is deliberately small in scope. It is not a web framework and it does not tell services how to be written. It is not a general scheduler for business logic, and it is not a data warehouse. It does not decide whether a release is a good idea — only whether it can be done safely and how it would be undone.

SECTION 1.2

What went wrong four weeks ago

One routine release broke eleven services for four hours and seventeen minutes. About 340,000 pieces of work were handled twice. Cleaning up afterwards took six days.

INC-2291 **Duplicate work everywhere, after one small release** SEVERITY 1

HOW LONG	4h 17m from the first symptom until things worked normally again, plus 6 more days of cleaning up data.
WHAT STARTED IT	A release of the inventory service that changed when it reported a piece of work as finished.
HOW FAR IT SPREAD	11 of 40 services degraded. 3 of them were completely down for a while. All background work was delayed.
CUSTOMER IMPACT	About 90,000 people received duplicate notifications. Order status was visibly wrong for 41 minutes. No data was lost for good.
HOW IT WAS FOUND	A customer support ticket, 38 minutes after the first symptom. The platform did not notice at all.

The change itself was small and correct on its own. The service used to report work as finished *before* saving it; the new version reported it *after*, which is the safer order. But that made the gap between receiving work and reporting it finished much longer, from milliseconds to several seconds. If the platform does not hear “finished” within that gap, it assumes the work was never done and sends it again. The workers started dying inside the gap.

Then the restart logic made things much worse. It restarted the dying workers straight away and never gave up. A worker that crashed on every start was restarted 70 times in 70 minutes, and each restart produced another batch of duplicate work. The restart logic did exactly what it had been told to do. What it had been told to do was wrong.

TIMELINE, COUNTED FROM THE MOMENT OF THE RELEASE

TIME	WHAT HAPPENED
+00:00	The release starts. Health checks pass.
+00:06	The release is marked successful. The new version now reports work as finished later than the old one did.
+00:07	Inventory workers start crashing on a case the new code does not handle. They are restarted. They crash again.
+00:14	The platform starts resending work. Services begin receiving the same work twice. By +00:23 customers are getting duplicate notifications, and nothing flags this as unusual.
+00:38	Customer support raises a ticket. This is the first time any human knows.
+00:52	Cached stock counts, updated twice by the duplicate work, no longer match the record of what was sold. Customers can see wrong order status.
+01:31	An operator finally connects the release to the crashing workers. Until now the two were shown in different places and did not look related.
+01:50	The release is undone by hand. Doing so meant rebuilding the old settings from a record that was never meant for that.
+02:11	Workers are stable. A backlog of 1.2M items starts clearing, and clearing it that fast slows down two services that were healthy.

TIME	WHAT HAPPENED
+04:17	Everything works normally again. Cleaning up the duplicated effects takes another six days.

WHERE THE FOUR HOURS ACTUALLY WENT

An engineer who looked at the problem directly understood it in nineteen minutes. Thirty-eight minutes were spent not knowing anything was wrong. Fifty-three minutes were spent working out that the release and the crashes were connected, because the platform stored those two facts separately and never linked them. Nineteen minutes were spent undoing the release by hand. So the platform not being able to explain itself cost about ninety minutes, not being able to undo a change cost another twenty, and the actual bug cost six.

SECTION 1.3

What the review found

Six findings. Each is something the platform cannot do today, and together they are the reason for the work in Part Four.

FINDING	WHAT IT MEANS
01 Restarting never stopped	Something that was broken in the same way every time kept being restarted forever. Restarting helps when a fault is temporary and hurts when it is permanent. The platform could not tell the two apart.
02 Delivery had no memory	The platform could not tell that a piece of work had already been sent before. Every service therefore had to cope perfectly with receiving the same work twice, and not all of them did.
03 Releases were not linked to their effects	Nothing in the records connected a new version going out to the strange behaviour that followed. Working out that link by hand took the largest single chunk of the incident.
04 Disagreements went unnoticed	Two parts of the system held different values for 41 minutes and nothing was able to spot it. A customer spotted it instead.
05 Undoing a release was a procedure, not a button	Taking the release back meant a human rebuilding the old state by hand, under pressure, having not done it recently.
06 Clearing the backlog had no speed limit	The 1.2M item backlog was cleared as fast as possible, which slowed down two services that had been fine all along. The recovery caused its own smaller incident.

PART TWO · WHAT YOU BUILD

SECTION 2.1

What you are building

You are building a small working model of NEXUS that runs on one machine. You are not rebuilding the 40 services. You build the platform that sits under them, plus a few simple stand-in services of your own, which is enough to show that the platform works.

FOUR THINGS MAKE UP YOUR SUBMISSION

THING 01 **The platform**	The main piece of work. It takes in work, hands it to workers, retries it, gives up on it in a controlled way, restarts things that die, and records what it did and why.
THING 02 **Stand-in services around it**	Small services you write yourself: something that sends work in, and one or more workers that process it. They can be very simple. What matters is that you can start them, stop them, and make them fail on purpose.
THING 03 **An operator view**	A screen an on-call engineer looks at. A web page or a terminal view are both fine. What it must show is listed below.
THING 04 **A way to break it on purpose**	A button, a command, or a flag that triggers each failure you say you handle. Reviewers will use this. Build it early, not at the end.

You decide what a “piece of work” looks like. Keep it simple. An id, a type, and a small body is enough. Nothing in this document depends on what is inside it.

WHAT THE OPERATOR VIEW MUST SHOW

- **What is wrong right now**, stated by the platform. Do not show raw numbers and leave a tired human to work out which ones are bad.
- **When it started.**
- **What changed just before that** — releases and operator actions, on the same timeline as everything else.
- **The state of the work**: how much is waiting, how old the oldest item is, and what has been given up on.
- **The state of each component**: running, restarting, or taken out of service, with how many times it has been tried.

Assume the reader was asleep ninety seconds ago, did not write the failing component, and may be on a phone. Do not build it for yourself: you know which numbers are normal, they do not.

FAILURES A REVIEWER WILL TRY TO CAUSE

- Kill a worker in the middle of processing a piece of work.
- Stop and restart your platform while it is holding work.
- Make a worker crash every time it starts, so it never recovers.
- Make a worker slow instead of dead.
- Cause the same piece of work to be delivered twice.
- Push out a bad release and then take it back.
- Make a cached value disagree with the real value.
- Take a dependency away completely.

You do not have to handle all of these. You do have to be honest about which ones you handle, and be able to trigger those on demand.

Size to design for. A few thousand pieces of work, a backlog of around ten thousand that must clear without harming healthy components, and a handful of workers. Not millions. The big numbers in Part One describe the real system; you are building a small model of it.

SECTION 2.2

Rules you cannot break

These rules are absolute. They exist so that someone who did not build your platform can run it, test it, and understand it, on a machine that is not yours, with no internet.

RULE 01 **It runs entirely on one machine**	Everything must run locally, with nothing outside it. A reviewer must be able to start it, use it, and watch it without accounts, passwords, or a network connection.
RULE 02 **No cloud services**	Nothing may depend on a hosted service from any provider, including AWS, Azure, and GCP. That covers hosted databases, hosted queues, hosted login, and hosted monitoring.
RULE 03 **No AI model calls while it runs**	The running platform must not ask a model to make a decision. Everything it does must follow from its inputs and its code. Use whatever tools you like while building; the thing you hand over must not.
RULE 04 **Same inputs, same behaviour**	Given the same inputs and the same saved state, your platform must make the same decisions. If something depends on time or randomness, that must be visible and controllable, so a failure can be reproduced.
RULE 05 **Understandable by using it**	A reviewer must be able to see what your platform is doing by using it, not by attaching a debugger or opening log files in a text editor.

● NO TECHNOLOGY IS PRESCRIBED

This document never names a database, a queue, a language, a framework, or a container system, and it will not. Choosing the tools is part of the work being reviewed. Use anything that fits the rules above, including nothing extra: a platform built only from what your language already gives you, storing state in files you manage yourself, is a perfectly good answer at this size. What is judged is not what you picked, but whether you can explain why you picked it.

SECTION 2.3

Requirements

Each requirement says *what must be true*, never *how to do it*. You meet a requirement when your platform behaves that way, by any means. Each one has a class:

CLASS	MEANING	IF YOU SKIP IT
CORE	Without it, this is not the platform.	Your submission is incomplete, and you must say so plainly.

CLASS	MEANING	IF YOU SKIP IT
EXPECTED	A good platform has it. Leaving it out should be a choice.	Fine, if you say so and explain why.
EXTENDED	Stands out, but only worth reaching once the rest is solid.	No penalty. Reaching for it too early is itself a mistake.

You will not do all of this in six hours. The classes are there to help you choose. Four core requirements done convincingly beat all fifteen done badly.

REF	REQUIREMENT	CLASS
R-01	**Accepted work is safe.** Once the platform accepts a piece of work, that work survives any single part of the platform dying, including the part that accepted it.	CORE
R-02	**Every piece of work ends somewhere.** Each accepted piece is either processed, or ends up in a state an operator can see and act on. There is no third outcome.	CORE
R-03	**Doing it twice is harmless.** Work delivered more than once has the same visible result as work delivered once, or the platform makes the repeat obvious to whoever receives it.	CORE
R-04	**Trying again has a limit.** Retries, restarts, and recovery attempts each have a stated limit. Hitting the limit produces a visible state, not an endless loop.	CORE
R-05	**You can ask about the past.** For any recent period, an operator can find out what the platform was asked to do, what it did, and why.	CORE
R-06	**Changes can be undone.** Any release the platform pushes out can be taken back in one action with a known result, without a human rebuilding the old state by hand.	CORE
R-07	**Changes are linked to what followed.** The records connect a release to the behaviour seen afterwards, closely enough that an operator does not have to match timestamps by eye.	EXPECTED
R-08	**Disagreements are found.** When two parts hold different values for the same thing, the platform notices within a stated time and reports it, instead of quietly fixing it.	EXPECTED
R-09	**Copied values carry their age.** Anything the platform caches carries how old it is, and the platform refuses to serve it past a stated limit.	EXPECTED
R-10	**Degrading is honest.** When something it depends on is unavailable, the platform still serves what it can and clearly marks what it cannot.	EXPECTED
R-11	**Recovery does no harm.** Clearing a backlog, replaying work, or bringing something back is speed-limited, so recovering cannot damage a part that was healthy.	EXPECTED
R-12	**A guess within ninety seconds.** From noticing something is wrong, your screens lead an engineer who did not build the broken part to a sensible first guess within ninety seconds.	EXPECTED
R-13	**Order is only claimed when known.** When the platform shows a sequence, it separates records whose order it actually knows from records that merely appear next to each other.	EXTENDED

REF	REQUIREMENT	CLASS
R–14	**The platform can be asked about itself.** An operator can ask what it currently believes, how old that belief is, and what would change it.	EXTENDED
R–15	**Failures can be triggered.** The platform gives you a deliberate way to cause the failures listed in Section 2.1, so its behaviour can be shown rather than claimed.	EXTENDED

Beyond these, anything run here is expected to start from cold with one documented command and reach a known state on its own; to be stoppable and restartable at any moment without losing accepted work or needing cleanup; to show the settings it is actually using while running; to fail loudly at startup when misconfigured rather than quietly later; and to use a known, stated amount of memory and CPU.

> **BUILD IN THIS ORDER**
>
> The requirements depend on each other. R-01, keeping accepted work safe, comes before almost everything: R-02 and R-05 need it, and R-03 and R-04 need those. Nothing in the Extended class is worth starting before the ones above it work. Building something that sits on top of a piece you have not built yet is the most common way to waste hours. R-12 is the only requirement measured in human time, and you can actually test it: sit someone else in front of your platform, break something, and time them.

PART THREE · WHAT THE PLATFORM MUST DO
SECTION 3.1

Moving work around reliably

The platform takes work from one service and is responsible for it from then on. Everything else in this part sits on top of this.

The real problem. A worker is down for eleven minutes. Forty thousand pieces of work pile up for it. When it returns it must get all of them, at a speed that does not knock it over again, and none may be lost. Worse: some may have been half-done before it died. Work that never arrived, work whose “done” message was lost, and half-finished work all look identical from outside.

WHAT IT MUST DO

- **Say clearly whether it took the work.** The sender always knows whether the platform accepted a piece of work. There must be no case where the sender thinks the platform has it and the platform does not.
- **Keep work across a restart.** Accepted work outlives the process that accepted it. Restarting mid-delivery must not change how that work ends up.
- **Retry slower each time, and stop eventually.** Do not hammer a worker that is already failing. Space out retries, and cap the total number.
- **Make dead-end work visible.** Work that stopped without finishing must appear somewhere an operator already looks, with enough detail to choose between retrying it and throwing it away. It must not vanish, and it must not retry forever.
- **Deal with repeats explicitly.** Either guarantee that receiving the same work twice changes nothing, or give the receiver a way to spot a repeat. Remembering what you have already seen costs storage, so state how long you remember and what happens past that.

- **Treat the backlog as a real thing.** You can count what is waiting, see how old it is, and see how fast it is growing. A backlog you only discover because things feel slow is not visible.

SITUATION	WHAT THE PLATFORM MUST NOT DO
A worker says “done” after the platform already resent the work	Accept that late message as final for work that is now running twice, leaving numbers an operator cannot make sense of.
A worker finishes the job but dies before saying “done”	Assume it never happened and send it again with no hint to the receiver that this might be a repeat.
The platform restarts while holding accepted work	Carry on delivering but forget how many attempts were already made, handing everything a fresh budget.
A worker is permanently slower than the sender	Pile work up until the machine falls over, instead of slowing the sender down or visibly dropping work.

SECTION 3.2

Releasing safely

The platform pushes new versions of services out, checks they are behaving, and takes them back when they are not. Change is where most incidents come from.

The real problem. A service is released and starts. Its health check passes, because health checks say whether a process is alive, not whether it is correct. It behaves slightly differently, and that only shows up two services away, ten minutes later, in a record kept somewhere else entirely. What matters most is not spotting a bad release fast. It is undoing it fast.

WHAT IT MUST DO

- **Know how to undo before doing.** The platform knows how it would take a release back before it pushes one out. If it cannot say how, it refuses.
- **Judge behaviour, not aliveness.** A release is judged on whether things are still behaving as before, not on whether a process answered a ping.
- **Watch for a while afterwards.** A release is not final the moment it starts. Some stated period follows where the platform is still watching and undoing is still cheap. “Not sure” is a real answer at the end of that period, and treating it as “fine” is how bad releases survive.
- **Undo in one action.** One action, with a known result, that an operator who has never done it before can perform under pressure without reading anything.
- **Control overlapping changes.** Two releases that could affect each other are not pushed at the same time unless someone deliberately chose that, and it was recorded.
- **Link the release to what happened next.** Someone looking at odd behaviour can see what changed shortly before it, without comparing two screens by eye.

SITUATION	WHAT THE PLATFORM MUST NOT DO
The new version starts, then fails during the watching period	Record the release as a success because starting worked.
Someone asks to undo while the release is still going out	Leave a mix of old and new versions running with no record of which is where.
The old version can no longer run, because things around it moved on	Find this out at the moment undoing is needed, instead of before releasing.
The problem only appears under load the watching period never saw	Treat a quiet watching period as proof that the release is good.

SECTION 3.3

Keeping copies of data in step

The same fact is stored in more than one place. The platform's job is to notice when those places disagree, and to make that disagreement visible.

The real problem. A stock count lives in the inventory service, in a cache in front of it, and in the record of what was sold. When work runs twice, the cache updates twice but the record rejects the repeat. Two places now hold different answers, both by working exactly as designed, and nothing is looking for that. Copies drifting apart is normal. Nobody noticing is not.

WHAT IT MUST DO

- **Compare within a stated time.** Any two copies of a fact get compared within a known period, so a disagreement has a maximum lifetime before someone knows.
- **Compare ages too.** Two values read at different times are not really comparable. Account for that instead of reporting a disagreement that is not one.
- **Decide per fact who is right.** Some facts have an owner and can be fixed automatically. Others have no owner and must be reported to a human. The platform should know which case it is in, and be able to do both.
- **Record a fix as an action.** If the platform fixes a disagreement, that is something it did, with a before, an after, and a reason — not a silent correction.
- **Escalate disagreements that keep coming back.** A difference that survives several checks is a different problem from a brief one, and should be treated as such.

SITUATION	WHAT THE PLATFORM MUST NOT DO
Both copies are out of date, and match each other	Report that everything agrees, which is technically true and completely misleading.

SITUATION	WHAT THE PLATFORM MUST NOT DO
A value is being changed while it is being compared	Report a disagreement for what is just a normal update in progress.
One side cannot be reached during the check	Treat no answer as agreement.
A disagreement is fixed automatically and comes straight back	Keep fixing it without noticing that repeated fixing is itself the warning sign.
The copy treated as correct is the wrong one	Confidently copy the wrong value everywhere and record it as a successful fix.

SECTION 3.4

Bringing things back after they die

Components die. The platform must bring them back when that helps, stop when it does not, and know the difference.

The real problem. A worker dies seven seconds after every start, because of a release. It is restarted. It dies again. Seventy minutes of that means seventy restarts, seventy half-processed batches, and seventy rounds of duplicate work. Every single step was correct; the overall behaviour was the incident. A temporary fault and a permanent one look identical for two attempts and obvious by the fifth.

WHAT IT MUST DO

- **Give restarts a budget.** A stated number of attempts within a stated period. Running out is an event, not a reason to keep going.
- **Wait longer each time.** Space attempts further apart, so a permanent fault costs less and less while it stays broken.
- **Make it earn "recovered".** Something counts as recovered only after working for a settling period, and the budget resets then, not the moment it starts. Starting is not recovering.
- **Make giving up visible and reversible.** Something taken out of service is shown clearly, stops receiving work, and can be put back by an operator without restarting the whole platform.
- **Do not flood on the way back.** Bringing something back must not release a wave of waiting work that damages whatever is currently healthy.
- **Hand over the history.** When the platform gives up, what it tells the operator includes what it tried, how often, and what happened each time.

SITUATION	WHAT THE PLATFORM MUST NOT DO
Something only fails once real work reaches it after a restart	Reset its budget on start, so it never runs out and loops forever.

SITUATION	WHAT THE PLATFORM MUST NOT DO
Many things fail at once because the thing they all use failed	Try to restart all of them at the same time against something that is still down.
The part doing the restarting is itself restarted	Forget the attempt history and give every failing component a fresh budget.
Something is failing slowly instead of dying	Ignore it, because the recovery logic only notices processes that have exited.

SECTION 3.5

Caching without lying

Keeping copies of expensive answers makes everything fast. It also makes it possible to be confidently wrong, which is worse than being slow.

The real problem. A cached value is updated by work that ran twice, while the real store rejects the repeat. The cache now holds a number the real store never produced, and everyone reading through it gets that number. Nothing crashed, nothing errored, nothing will notice. A fast wrong answer is easy. The skill is an answer that is either right or clearly marked as maybe-not.

WHAT IT MUST DO

- **Every cached value carries its age**, and whoever uses the value can see that age, not just the cache itself.
- **State how old is too old.** Each cached fact has a maximum age, chosen deliberately for that fact rather than one global default.
- **Serving old data must be a stated choice.** Going past the limit is allowed when the real source is unreachable, as long as the reader is told. Never silently.
- **Clear the copy when the real thing changes**, rather than waiting for it to expire and hoping. Always keep a maximum age as well: clearing messages get lost.
- **One refresh, not a hundred.** Many readers missing the same value at the same moment cause one refresh. Expiry must not turn into a traffic spike.
- **Be able to check the cache.** There is a way to ask whether a cached value still matches the real one, without waiting for someone to read it.

SITUATION	WHAT THE PLATFORM MUST NOT DO
The real source is down and everything cached has expired	Turn a slow dependency into a total outage by refusing to answer at all.
A “this changed” message is lost	Rely only on those messages, with no maximum age as a safety net.

SITUATION	WHAT THE PLATFORM MUST NOT DO
Something writes to the cache without going through the real source	Allow it, so the cache can hold values the real source never produced.

SECTION 3.6

Recording things so a human can understand them

The platform records what it did and shows it in a way a stressed human can use. This is what INC-2291 was really about.

The real problem. An operator is woken up. They get about ninety seconds of clear thinking before the first “any update?” arrives. In that window they need three answers: what is wrong, when did it start, what changed around then. This is not a request for more data. During INC-2291 everything needed was already recorded. The records just sat apart, referred to nothing, and left a human joining them up by eye. That cost fifty-three minutes.

WHAT IT MUST DO

- **Record why, not just what.** When the platform does something on its own, record what it believed at the time and why that justified the action.
- **Group records by subject.** Everything about one piece of work, one component, or one release can be pulled up together, without a human matching timestamps.
- **Put changes on the same timeline as effects.** Releases and operator actions appear next to platform behaviour, because “what changed just before this?” is the question that solves most incidents.
- **Say what is wrong, do not imply it.** State plainly what the platform thinks is abnormal rather than showing raw numbers and leaving the judgement to someone at 3am.
- **State how far back you can ask.** The period you keep records for is known, chosen, and enforced, and an operator knows what it is.
- **The record must survive the failure.** Whatever explains a breakage must not be the thing that breaks with it.

SITUATION	WHAT THE PLATFORM MUST NOT DO
Recording volume spikes during an incident	Slow down the platform it is supposed to explain, or quietly drop exactly the records that matter.
Someone asks about a time you no longer keep	Return an empty result that looks identical to “nothing happened”.
Every check passes, but throughput has halved	Report that everything is healthy, because no single check failed.
Parts of the platform itself fail	Show a screen that looks exactly like a quiet, healthy system.

Design the operator screen first, then build the recording that makes it possible. Doing it the other way round reliably produces a complete archive that answers no question anyone has.

PART FOUR · THE ASSIGNMENT

SECTION 4.1

What you are asked to do

Build the part of NEXUS you think matters most, to a standard you would be happy to run yourself, and be honest about everything you did not build.

That sentence is the whole task. In practice you hand over the four things listed in Section 2.1, showing some of the behaviour described in Parts Two and Three, plus a short written account of your decisions. All of it is reviewed. The written account is not documentation you add at the end. It is the part that shows how you think.

You have about six hours and you will not finish. Decide now what unfinished looks like. There is no minimum number of features: a platform that reliably holds on to work, survives its own restart, and explains what it did is a complete answer. One that touches all six areas in Part Three and survives no failure is not.

HOW TO CHOOSE WHAT TO BUILD

- **Follow the order in Section 2.3.** Building on top of a piece you have not built yet is the most common way to waste hours.
- **Go deep in one place, not shallow in six.** One area done properly, including what breaks and what the operator sees, shows far more than six that only work when nothing goes wrong. The interesting engineering is in the failure handling.
- **Pick things you can show.** Something you can trigger in front of a reviewer in under a minute beats several you can only describe. Build the break-it-on-purpose controls early.
- **Write your plan down first**, including what you are leaving out. It is what stops you starting a seventh thing instead of finishing the third.

SECTION 4.2

What “finished” means here

- **It runs.** One documented command on a clean machine, and it reaches a known state on its own.
- **It survives.** Stopping and restarting it loses no accepted work and needs no cleanup.
- **It fails the way you said.** The failures you claim can be triggered, and it behaves as described. Shown, not claimed.
- **It explains itself.** Someone who did not build it reaches a sensible guess within ninety seconds.
- **Its edges and decisions are written down**, with the alternatives you rejected, so gaps are declared rather than discovered.

SECTION 4.3

What to hand over

Three things, reviewed in the reverse order of the effort most people put into them.

ITEM 01 **The platform**	The code and everything needed to run it, following Section 2.2. It must start on a machine that is not yours.
ITEM 02 **The written account**	Two pages of specifics beat ten of description. Six headings. **Scope**: what you built and what you deliberately left out. **Decisions**: what you chose, what you rejected, what would change your mind. **Failure behaviour**: which failures you handle, what happens in each case, how a reviewer triggers them. **Limits**: where it stops working and what happens there. **Confidence**: what you tested, what you only reasoned about, what you assumed. **Next**: what another six hours would do, and in what order.
ITEM 03 **How to run it**	Written for someone who has never seen your platform, will run it in the next four minutes, and will not scroll past the first screen. **Start**: the one command that brings it up, what success looks like, and anything needed first. **Use**: how to push work through it and watch it being handled, within two minutes. **Break**: how to trigger each failure you claim to handle, and what to watch. **Look**: where the operator view is and what to notice first.

● IMPORTANT

A platform that does not start from your own instructions is treated as a platform that does not start. Reviewers will not debug your setup. In your last fifteen minutes, start it from scratch following only your own instructions, stop and restart it while it holds work, trigger every failure your account claims, and check nothing touches the network. The *Break* section is the one reviewers use most and the one most often missing. If you write only one section well, write that one.

Then reread the account against what you built, and fix anything it implies but the platform does not do. Last, look at your operator view as though you had just woken up: if you cannot tell what is wrong, neither can a reviewer.

How it is reviewed. Reviewers read your account, then run your platform on a machine that is not yours, break it, and ask about your decisions and what you left undone. A gap you predicted costs almost nothing. A gap you did not predict puts every other claim in doubt. Nobody is marked down for building less than they wanted to.

SECTION 4.4

How to submit

Deadline and where to send it	Shared separately.
What to send	One repository link or one archive, named `nexus-yourname`, holding the code, a top-level `README.md` with the run instructions from Section 4.3, and `ACCOUNT.md` with the written account.
What to leave out	Downloaded dependency folders, build output, and anything holding a password or a key. If a reviewer needs it, say so in `README.md` instead of shipping it.
After the deadline	Stop changing it. Reviewers use exactly what was submitted when time ran out.

Before sending, copy your submission into an empty folder, open only your own `README.md`, and follow it exactly. What happens there is what happens in review.